
GramVest

SIH 2026 • PS 26091

M1 - Frontend & User Journey

Module implementation handbook - frontend owner

TEAM IMPLEMENTATION HANDBOOK

Prepared as a practical engineering guide for independent module development, stable integration, testing, and handoff.

Source basis: SIH 26091 Implementation Blueprint and the corresponding GramVest module Markdown handbook.

Contents

1	1. Mission	1
2	2. Why this module exists	1
3	3. Scope	1
3.1	In scope	1
3.2	Out of scope	1
4	4. Recommended tools	2
5	5. Frontend repository structure	2
6	6. Design system to freeze	3
7	7. Main pages	3
7.1	A. Landing	3
7.2	B. Onboarding	3
7.3	C. Dashboard	4
7.4	D. Market	4
7.5	E. Competitors	4
7.6	F. Opportunity	4
7.7	G. Risks	4
7.8	H. Money	4
7.9	I. Financing	4
7.10	J. Advisor	4
7.11	K. What-If	4
7.12	L. Report	4
8	8. Frontend API usage pattern	5
9	9. Shared domain object	5
10	10. Frontend component contract rules	5
11	11. Integration steps	5
11.1	Step 1	5
11.2	Step 2	5
11.3	Step 3	6
11.4	Step 4	6
11.5	Step 5	6
11.6	Step 6	6
11.7	Step 7	6
11.8	Step 8	6
11.9	Step 9	6
11.10	Step 10	6
12	12. Loading / empty / error states	6
13	13. Test checklist	6
14	14. Definition of done	7
15	15. Handoff to other modules	7
16	16. Common mistakes to avoid	7

1. Mission

Build the complete user-facing GramVest application and make the backend capabilities understandable to a rural micro-entrepreneur.

The frontend is responsible for **presentation and interaction**, not business calculations. It consumes verified API results from the backend and renders forms, maps, charts, recommendations, finance summaries, scenarios, AI answers and reports.

The blueprint specifies Next.js/React, Tailwind CSS, Recharts, and Leaflet + OpenStreetMap for the prototype; the frontend screen plan includes Landing, Entrepreneur Profile, Market Intelligence, Competitor Map, Business Score, Financial Simulator, Scheme Router, AI Advisor, What-If, and Final Report.

2. Why this module exists

The platform has many sophisticated engines, but the target user should not have to understand the engines. M1 turns complex output into a guided journey:

User → Profile → Market → Opportunity → Risk → Business Result → Money → Financing → Advisor → What-If → Report

The user should always know:

- what this page is about
- what the result means
- what to do next

3. Scope

In scope

- public landing page
- language selection
- registration/onboarding UI
- entrepreneur profile forms
- business/location forms
- dashboard
- market view
- competitor map UI
- opportunity page
- risk/SWOT presentation
- viability presentation
- money/financial UI
- financing/scheme UI
- advisor chat UI
- What-If controls and comparison UI
- report preview/download UI
- loading/empty/error states
- responsive design

Out of scope

- EMI calculation logic
- DSCR calculation logic
- PostGIS calculations
- scheme rule evaluation
- RAG retrieval
- LLM generation

These are backend/module responsibilities. Frontend only displays their outputs.

4. Recommended tools

Task	Tool
Coding	VS Code
UI	Next.js + React + Tailwind CSS
Components	shadcn/ui or another agreed component library
Icons	Lucide
Charts	Recharts
Maps	Leaflet + OpenStreetMap
API calls	native fetch or Axios; choose one and standardize
Forms	React Hook Form + Zod if adopted by team
API testing	Postman/Insomnia
Version control	Git + GitHub
Design reference	Stitch/Figma exports/screens
Local environment	Docker only where needed

Do not introduce a new library for a small UI requirement unless the team agrees.

5. Frontend repository structure

```
frontend/
├── app/
│   ├── page.tsx
│   ├── onboarding/
│   ├── dashboard/
│   ├── market/
│   ├── opportunity/
│   ├── money/
│   ├── financing/
│   ├── advisor/
│   ├── scenarios/
│   └── reports/
├── components/
│   ├── layout/
│   ├── navigation/
│   ├── forms/
│   ├── market/
│   ├── finance/
│   ├── risk/
│   ├── advisor/
│   ├── scenarios/
│   └── reports/
├── services/
│   ├── profile.ts
│   ├── market.ts
│   ├── opportunity.ts
│   ├── finance.ts
│   ├── schemes.ts
│   ├── advisor.ts
│   ├── reports.ts
│   └── scenarios.ts
└── types/
```

```
|   └─ api.ts
|   └─ domain.ts
└─ hooks/
└─ lib/
```

6. Design system to freeze

Use the approved GramVest visual identity:

- warm ivory background
- charcoal text
- terracotta primary action
- earthy green for positive/healthy states
- warm yellow/ochre for attention
- minimal borders and shadows
- Manrope for UI
- no blue/purple AI gradients
- no glassmorphism
- no neon

Navigation:

HOME

Dashboard

YOUR BUSINESS

Market

Opportunity

Money

Financing

GET HELP

Advisor

What if?

YOUR REPORT

Feasibility Report

Global top bar should remain quiet:

- business name
- location
- profile/help

Do not put the full 01 IDEA → 05 REPORT workflow in the permanent global top bar.

7. Main pages

A. Landing

Goal: explain value and start analysis.

Primary action: Start my business check.

B. Onboarding

Collect only the necessary profile/business inputs with a guided questionnaire. The blueprint's required examples are location, own capital, business category/idea, experience, with optional target customers, existing assets, scale, income and risk preference.

C. Dashboard

Show the business conclusion first:

- overall result
- market
- money
- customers
- risk
- next best action

D. Market

Show:

- analysis radius
- potential customers
- households
- competitors
- map
- market signals
- 2–3 human-readable observations

E. Competitors

Show:

- map
- categories
- distance
- competitor list
- source/date note

F. Opportunity

Lead with a conclusion, then evidence and map.

G. Risks

Lead with the biggest risk and its mitigation; show SWOT as a secondary view.

H. Money

Show project cost, own contribution, financing gap, monthly revenue, expenses, surplus and repayment result. Hide technical finance details behind an expandable area.

I. Financing

Show strongest indicative route first, alternatives second, documents and comparison after that.

J. Advisor

Show plain-language answer, evidence, numbers, recommendation and optional calculation/source details.

K. What-If

Support Quick Test and Advanced modes. Quick Test should provide common scenarios; Advanced mode can expose sliders.

L. Report

Default to a simple feasibility view. Detailed DPR can be an advanced view.

8. Frontend API usage pattern

The frontend should never calculate authoritative business numbers.

Example:

```
const result = await financeApi.calculateProject(payload);
setFinance(result.data);
```

Render:

```
<MonthlySurplus value={finance.monthly_surplus} />
```

Do not do this in a component:

```
const emi = loan * rate; // WRONG: authoritative calculation belongs to M4/backend
```

9. Shared domain object

Consume the team's agreed BusinessAnalysisContext.

Suggested shape:

```
interface BusinessAnalysisContext {
  profile: Profile;
  location: LocationContext;
  market: MarketSummary;
  competitors: CompetitorSummary;
  opportunity: OpportunityResult;
  risks: RiskResult;
  viability: ViabilityResult;
  finance: FinanceResult;
  schemes: SchemeResult;
  advisor?: AdvisorContext;
  scenarios?: Scenario[];
  report?: ReportMetadata;
}
```

Do not invent competing versions of this interface.

10. Frontend component contract rules

Every reusable component should have:

- typed props
- loading state
- empty state where relevant
- error state where relevant
- accessible labels
- mobile behavior

Example:

```
FinancialSummary
  input: FinanceResult
  output: UI summary
```

11. Integration steps

Step 1

Create layout/navigation from the approved UI.

Step 2

Create mock JSON matching the final API contracts.

Step 3

Complete the pages using mock data.

Step 4

Freeze shared component names and data types.

Step 5

Connect profile APIs.

Step 6

Connect market/opportunity APIs.

Step 7

Connect finance/scheme APIs.

Step 8

Connect advisor/scenario/report APIs.

Step 9

Remove mocks.

Step 10

Run end-to-end test from onboarding to report.

12. Loading / empty / error states

Loading:

- Checking your location...
- Looking at nearby businesses...
- Preparing your numbers...

Do not expose API jargon.

Empty competitor data:

We could not identify enough local competitor data. This does not mean there are no competitors.

AI missing data:

I do not have enough verified information to answer that reliably.

13. Test checklist

- all pages route correctly
- all buttons have a defined action
- all APIs handle loading/error states
- map works on desktop/mobile
- tables do not break mobile layout
- financial values are display-only
- scenario results replace old results cleanly
- report download works
- language selector does not break layout
- no hard-coded secrets

14. Definition of done

M1 is done when:

- all screens exist
- APIs are connected
- common types are used
- no authoritative calculations exist in UI code
- mobile/desktop states work
- loading/empty/error states exist
- end-to-end user journey works
- screenshots match the final design

15. Handoff to other modules

M1 provides:

- exact fields it needs from each API
- endpoint names
- UI screenshots
- UI acceptance criteria
- mock data used during development

M1 receives:

- backend API schemas from M2
- market data shape from M3
- finance result shape from M4
- advisor/report response shape from M5

16. Common mistakes to avoid

- calculating EMI in React
- hard-coding competitor data in components
- putting scheme rules in UI conditionals
- calling the LLM directly from the browser
- using different field names across pages
- adding a new library for every component
- hiding errors

17. Final user journey

Landing

→ Onboarding

→ Dashboard

→ Market

→ Opportunity

→ Risks

→ Money

→ Financing

→ Advisor

→ What-If

→ Report

The blueprint emphasizes one complete journey over disconnected features; frontend should make that journey feel continuous.